

SIH 2026 – Multispectral Sensor XGBoost Model

24 Spectral Input Channels

1 Introduction

This project implements a machine learning pipeline for classification using multispectral sensor data. The proposed model uses **XGBoost (Extreme Gradient Boosting)** as the classification algorithm.

The system considers 24 sensor input channels. Each channel contains 256 spectral measurements for every sample. Therefore, the raw input data has the following structure:

$$\text{Samples} \times \text{Channels} \times \text{Spectrum Length}$$

For the demonstration dataset:

$$1000 \times 24 \times 256$$

The raw spectral signals are converted into statistical and spectral features before being supplied to the XGBoost classifier.

2 Sensor Input Channels

The model uses the following 24 sensor channels:

Table 1: Multispectral sensor input channels

No.	Sensor Channel
1	B1x
2	B1y
3	B1z
4	B2x
5	B2y
6	B2z
7	dBx
8	dBy
9	dBz
10	magnetic_magnitude
11	acoustic_TOF
12	acoustic_amplitude
13	acoustic_backscatter
14	CSEM_voltage
15	CSEM_current
16	electric_field
17	pressure
18	temperature
19	ax
20	ay
21	az
22	angular_velocity_x
23	angular_velocity_y
24	angular_velocity_z

3 Python Implementation

The complete Python implementation used for the demonstration XGBoost pipeline is given below.

```

1
2 # =====
3 # SIH 2026 - Multispectral Sensor XGBoost Model
4 # 24 Spectral Input Channels
5 # =====
6
7 import numpy as np
8 import pandas as pd
9 import matplotlib.pyplot as plt
10
11 from sklearn.model_selection import train_test_split
12 from sklearn.metrics import (
13     accuracy_score,
14     classification_report,
15     confusion_matrix
16 )
17
18 from xgboost import XGBClassifier
19
20
21 # =====
22 # 1. SENSOR INPUT CHANNELS
23 # =====
24
25 SENSOR_CHANNELS = [
26

```

```

27     "B1x",
28     "B1y",
29     "B1z",
30
31     "B2x",
32     "B2y",
33     "B2z",
34
35     "dBx",
36     "dBy",
37     "dBz",
38
39     "magnetic_magnitude",
40
41     "acoustic_TOF",
42     "acoustic_amplitude",
43     "acoustic_backscatter",
44
45     "CSEM_voltage",
46     "CSEM_current",
47     "electric_field",
48
49     "pressure",
50     "temperature",
51
52     "ax",
53     "ay",
54     "az",
55
56     "angular_velocity_x",
57     "angular_velocity_y",
58     "angular_velocity_z"
59 ]
60
61
62 # =====
63 # 2. CONFIGURATION
64 # =====
65
66 N_SAMPLES = 1000
67 SPECTRUM_LENGTH = 256
68 RANDOM_STATE = 42
69
70
71 # =====
72 # 3. CREATE DEMONSTRATION DATA
73 # =====
74
75 print("\nGenerating demonstration dataset...")
76
77 np.random.seed(RANDOM_STATE)
78
79 X_raw = np.random.randn(
80     N_SAMPLES,
81     len(SENSOR_CHANNELS),
82     SPECTRUM_LENGTH
83 )
84
85 print("Raw input shape:")
86 print(X_raw.shape)
87
88
89 # =====
90 # 4. CREATE DEMONSTRATION TARGET
91 # =====
92
93 y = np.random.randint(
94     0,
95     3,
96     N_SAMPLES
97 )
98
99 print("\nTarget shape:")

```

```

100 print(y.shape)
101
102
103 # =====
104 # 5. SPECTRAL FEATURE EXTRACTION
105 # =====
106
107 def extract_features(X):
108     """
109     Convert each spectrum into statistical and
110     spectral features.
111
112     Input:
113         X = samples x channels x spectrum_length
114
115     Output:
116         Feature matrix
117     """
118
119     features = []
120
121     for sample in X:
122
123         sample_features = []
124
125         for channel in sample:
126
127             # Basic statistical features
128
129             mean_value = np.mean(channel)
130
131             std_value = np.std(channel)
132
133             min_value = np.min(channel)
134
135             max_value = np.max(channel)
136
137             rms_value = np.sqrt(
138                 np.mean(channel ** 2)
139             )
140
141             median_value = np.median(channel)
142
143             energy = np.sum(channel ** 2)
144
145             # Peak information
146
147             peak_index = np.argmax(
148                 np.abs(channel)
149             )
150
151             peak_value = channel[peak_index]
152
153             # Spectral centroid
154
155             magnitude = np.abs(channel)
156
157             frequencies = np.arange(
158                 len(channel)
159             )
160
161             if np.sum(magnitude) != 0:
162
163                 spectral_centroid = (
164                     np.sum(
165                         frequencies * magnitude
166                     )
167                     /
168                     np.sum(magnitude)
169                 )
170
171             else:

```

```

173         spectral_centroid = 0
174
175         # Add features
176
177         sample_features.extend([
178             mean_value,
179             std_value,
180             min_value,
181             max_value,
182             rms_value,
183             median_value,
184             energy,
185             peak_value,
186             peak_index,
187             spectral_centroid
188         ])
189
190         features.append(sample_features)
191
192     return np.array(features)
193
194
195     # =====
196     # 6. EXTRACT FEATURES
197     # =====
198
199     print("\nExtracting spectral features...")
200
201     X_features = extract_features(X_raw)
202
203     print("Feature matrix shape:")
204     print(X_features.shape)
205
206
207     # =====
208     # 7. TRAIN / TEST SPLIT
209     # =====
210
211     X_train, X_test, y_train, y_test = train_test_split(
212         X_features,
213         y,
214         test_size=0.20,
215         random_state=RANDOM_STATE,
216         stratify=y
217     )
218
219     print("\nTraining samples:")
220     print(X_train.shape[0])
221
222     print("Testing samples:")
223     print(X_test.shape[0])
224
225
226     # =====
227     # 8. CREATE XGBOOST MODEL
228     # =====
229
230     model = XGBClassifier(
231         n_estimators=300,
232         max_depth=6,
233         learning_rate=0.05,
234         subsample=0.8,
235         colsample_bytree=0.8,
236         objective="multi:softprob",
237         num_class=3,
238         eval_metric="mlogloss",
239         random_state=RANDOM_STATE
240     )
241
242
243     # =====
244     # 9. TRAIN MODEL
245     # =====

```

```

246
247 print("\nTraining XGBoost...")
248
249 model.fit(
250     X_train,
251     y_train
252 )
253
254 print("Training completed.")
255
256
257 # =====
258 # 10. PREDICTION
259 # =====
260
261 y_pred = model.predict(X_test)
262
263
264 # =====
265 # 11. MODEL ACCURACY
266 # =====
267
268 accuracy = accuracy_score(
269     y_test,
270     y_pred
271 )
272
273 print("\n=====")
274 print("MODEL PERFORMANCE")
275 print("=====")
276
277 print(
278     f"Accuracy: {accuracy * 100:.2f}%"
279 )
280
281
282 # =====
283 # 12. CLASSIFICATION REPORT
284 # =====
285
286 print("\nClassification Report:")
287
288 print(
289     classification_report(
290         y_test,
291         y_pred
292     )
293 )
294
295
296 # =====
297 # 13. CONFUSION MATRIX
298 # =====
299
300 cm = confusion_matrix(
301     y_test,
302     y_pred
303 )
304
305 print("\nConfusion Matrix:")
306
307 print(cm)
308
309
310 # =====
311 # 14. FEATURE IMPORTANCE
312 # =====
313
314 importance = model.feature_importances_
315
316 indices = np.argsort(
317     importance
318)[-20:]

```

```

319 plt.figure(
320     figsize=(10, 6)
321 )
322
323 plt.barh(
324     range(len(indices)),
325     importance[indices]
326 )
327
328 plt.xlabel(
329     "Feature Importance"
330 )
331
332 plt.ylabel(
333     "Feature Index"
334 )
335
336 plt.title(
337     "XGBoost Feature Importance"
338 )
339
340 plt.tight_layout()
341
342 plt.savefig(
343     "results_feature_importance.png",
344     dpi=300
345 )
346
347 plt.show()
348
349
350 # =====
351 # 15. SAVE MODEL
352 # =====
353
354 model.save_model(
355     "xgboost_sih_model.json"
356 )
357
358 print(
359     "\nModel saved as: "
360     "xgboost_sih_model.json"
361 )
362
363
364 # =====
365 # 16. SAVE PREDICTIONS
366 # =====
367
368 results = pd.DataFrame({
369     "Actual": y_test,
370     "Predicted": y_pred
371 })
372
373 results.to_csv(
374     "xgboost_predictions.csv",
375     index=False
376 )
377
378 print(
379     "Predictions saved as: "
380     "xgboost_predictions.csv"
381 )
382
383
384 print("\n=====")
385 print("XGBOOST PIPELINE COMPLETED")
386 print("=====")
387

```

Listing 1: SIH 2026 multispectral sensor XGBoost implementation

4 Feature Extraction

Each of the 24 sensor channels contains 256 spectral values. Instead of directly providing all spectral points to XGBoost, the program extracts ten descriptive features from every channel.

The extracted features are:

1. Mean
2. Standard deviation
3. Minimum value
4. Maximum value
5. Root Mean Square (RMS)
6. Median
7. Energy
8. Peak value
9. Peak index
10. Spectral centroid

Therefore, the total number of features is:

$$24 \times 10 = 240$$

Hence, the final feature matrix has the approximate shape:

$$1000 \times 240$$

for the demonstration dataset.

5 XGBoost Classification

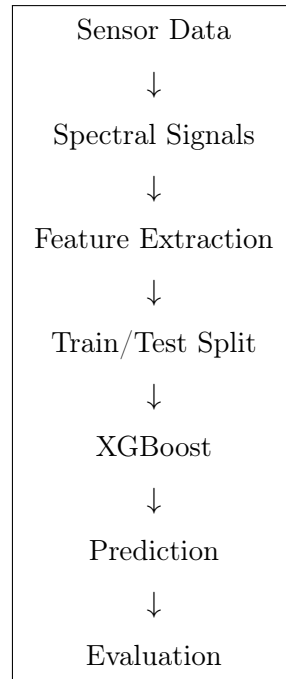
The extracted features are supplied to an XGBoost classifier. The important model parameters used in the implementation are:

Table 2: XGBoost configuration

Parameter	Value
Number of estimators	300
Maximum tree depth	6
Learning rate	0.05
Subsample	0.8
Column sampling	0.8
Objective	multi:softprob
Number of classes	3
Evaluation metric	mlogloss
Random state	42

6 Machine Learning Pipeline

The complete workflow can be represented as:



The main stages are:

1. Acquire multispectral sensor measurements.
2. Organize the measurements into samples, channels, and spectral points.
3. Extract statistical and spectral features.
4. Divide the dataset into training and testing subsets.
5. Train the XGBoost classifier.
6. Predict the class of unseen test samples.
7. Calculate classification accuracy and other metrics.
8. Generate the confusion matrix.
9. Analyze feature importance.
10. Save the trained model and predictions.

7 Important Note

The dataset generated in the current implementation is a **demonstration dataset** created using random values.

The following section:

```
1 X_raw = np.random.randn(  
2     N_SAMPLES,  
3     len(SENSOR_CHANNELS),  
4     SPECTRUM_LENGTH  
5 )
```

will be replaced by the actual multispectral sensor data when it becomes available.
Similarly, the demonstration target:

```
1 y = np.random.randint(0, 3, N_SAMPLES)
```

will be replaced by the actual class labels.

8 Generated Outputs

The program generates the following outputs:

- XGBoost trained model: `xgboost_sih_model.json`
- Prediction file: `xgboost_predictions.csv`
- Feature importance plot: `results_feature_importance.png`
- Classification report
- Confusion matrix
- Overall classification accuracy

9 Conclusion

The proposed pipeline converts multispectral sensor measurements into a compact feature representation and uses XGBoost for multi-class classification. The approach provides a complete workflow from sensor data preprocessing to model training, prediction, evaluation, feature importance analysis, and model storage.